

Autonomous Guided Vehicle Project

PMDS x DevNut Team

July 13, 2026

Abstract

This report documents the design and development of an Autonomous Guided Vehicle (AGV) system in a simulated warehouse environment. It describes the current controller, earlier controller iterations, the logging infrastructure and the web application used to inspect simulation results. Replace this text with a concise summary of the final system, its main design choices and the outcomes of the evaluation.

Contents

1	Current Controller	2
1.1	Architecture	2
1.2	Planning and Control	2
1.3	Observed Behaviour	2
2	Previous Controller Iterations	2
2.1	Initial Approach	2
2.2	Lessons Learned	2
3	Logging Infrastructure	2
3.1	Service Architecture	3
3.2	Data Model	3
3.3	Data Flow and Controller Integration	5
3.4	Validation and Transactional Integrity	6
3.5	JSONL Mirror and Dashboard Integration	7
3.6	Reliability Considerations	7
3.7	Use in Evaluation	8
4	Web Application	8
4.1	Interface Design	8
4.2	Data Integration	8
4.3	Practical Value and Limitations	8

1 Current Controller

This section describes the current AGV controller implemented for the Webots warehouse simulation. The controller integrates perception, localization, planning and low-level motion control to move each vehicle through the environment while accounting for configured goals and dynamic obstacles.

1.1 Architecture

Describe the controller as a sequence of cooperating components. Start from the information received from Webots and explain how sensor data is transformed into a state estimate, a route and finally motor commands. Refer to implementation details only when they clarify an engineering choice or a relevant limitation.

1.2 Planning and Control

Explain the high-level and low-level planning responsibilities, including how the controller selects waypoints, reacts to blocked paths and completes a goal. Use equations only when they make an algorithm or a measurable quantity clearer; otherwise, prefer a precise prose explanation.

1.3 Observed Behaviour

Summarize the behaviour observed during representative simulations. State what worked reliably, identify situations that required special handling and connect each limitation to its likely technical cause.

2 Previous Controller Iterations

The current controller was preceded by earlier implementations that helped establish the project requirements and expose practical limitations. This section records their main design choices so that the evolution of the system remains traceable.

2.1 Initial Approach

Describe the responsibilities and operating assumptions of the first controller. Focus on the motivation for the approach, the parts that were effective and the conditions under which it no longer scaled to the warehouse scenario.

2.2 Lessons Learned

Compare the earlier approach with the current controller in terms of robustness, maintainability and behaviour around dynamic obstacles. Present limitations as engineering observations rather than as unsupported claims, and explain how they informed subsequent design decisions.

3 Logging Infrastructure

The logging subsystem provides a persistent record of simulations, events and telemetry generated by the AGVs. Its role is to make the execution of a simulation inspectable after the fact and to provide the data consumed by monitoring tools. The pipeline is built around a client-server architecture: each robot controller collects events in memory and periodically flushes them to a standalone HTTP service backed by MySQL.

3.1 Service Architecture

The logging pipeline follows a client-server model. Each Webots robot controller process runs an instance of the `RobotLog` class in `logging/logger/robot_log.py`, which acts as an in-memory event collector and HTTP client. A separate process hosts the logging server defined in `logging/logging_server.py`, which exposes three HTTP endpoints served by Python's `ThreadingHTTPServer`. Using a thread-per-request model allows the server to handle concurrent flush operations from the two robots deployed in the warehouse world without blocking either controller.

The three endpoints serve distinct roles in the simulation lifecycle:

- `POST /start` accepts a `unit_id` string, creates a new row in the `Simulations` table, and returns the assigned `sim_id` to the client. This identifier anchors every subsequent event and telemetry record for the run.
- `POST /save` accepts a batch payload containing updated summary fields, a list of events and a list of telemetry records. The server validates the payload, opens a database transaction, and atomically writes all three components to MySQL. An HTTP 200 response indicates success; a 400 or 500 response signals validation or infrastructure failure, respectively.
- `GET /health` returns `{"status": "success"}` and serves as a liveness probe for the caller.

The server listens on `127.0.0.1:8080` by default. Command-line arguments `-host` and `-port` override the listener address, while the `LOGGING_SERVER_URL` environment variable controls the URL used by the robot controllers. This layered configuration supports local development, where the server and both controllers run on the same machine, and any future extension to a distributed setup.

3.2 Data Model

The server persists simulation data in three MySQL tables created by the queries in `Database_Structure.sql`. Each table addresses a distinct level of granularity: run-level aggregates, discrete event records, and per-event controller state snapshots. Figure 1 summarises the three tables, their columns and the relationships between them; the remainder of this subsection describes each table in turn.

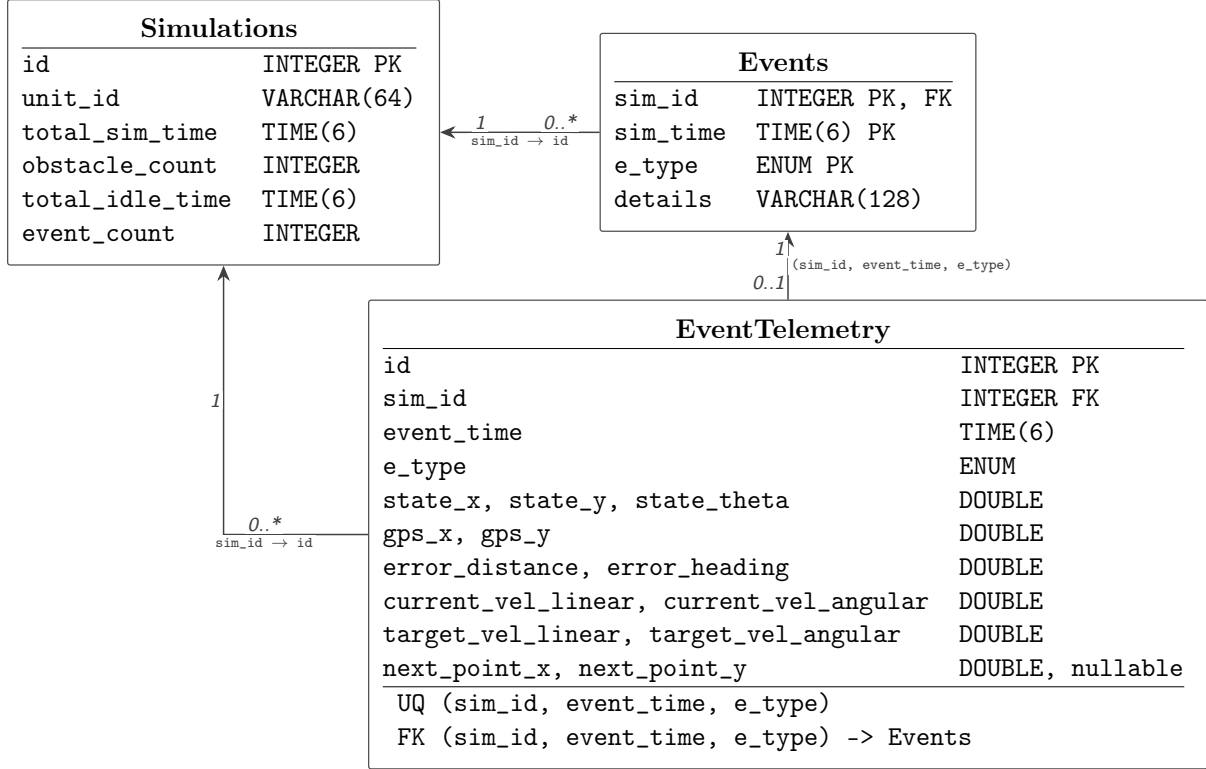


Figure 1: UML-style schema of the logging database. Each arrow points from a foreign key in the child table to the key that it references in the parent table. Multiplicities describe the number of child rows associated with one parent row. PK, FK and UQ denote primary-key, foreign-key and unique constraints, respectively.

Simulations. The **Simulations** table holds one row per completed run. Its auto-increment **id** serves as the primary key and is the foreign key target of the event-related tables. Each row stores the robot’s **unit_id** as a variable-length string and four aggregate metrics: **total_sim_time**, **obstacle_count**, **total_idle_time** and **event_count**. Temporal columns use the MySQL **TIME(6)** type with microsecond precision, preserving sub-second resolution for runs that span tens of minutes.

Events. The **Events** table records discrete state transitions and notable occurrences. Its composite primary key (**sim_id**, **sim_time**, **e_type**) ensures that a simulation cannot contain two events of the same type at the same instant. The **e_type** column is modeled as a MySQL **ENUM** restricted to eight values:

- **START** — logged when the controller confirms a new simulation record.
- **STOP** — logged during controller shutdown.
- **IDLE_START** and **IDLE_END** — mark transitions when the robot’s linear velocity crosses a near-zero threshold.
- **OBSTACLE_ENCOUNTER** and **OBSTACLE_CLEARED** — reflect the presence or absence of dynamic obstacles detected by the low-level safety module.
- **REACHED_TARGET** — recorded when the controller’s distance to the current goal falls below the configured threshold.
- **UNEXPECTED_BEHAVIOR** — an open-ended category for anomalous conditions observed by the controller.

Each event carries a `details` column of up to 128 characters. For obstacle events the details encode the GPS coordinates at which the obstacle was detected or cleared; for idle transitions they record the linear and angular velocity that triggered the state change; for target events they include the goal index and coordinates. This compact textual context makes the event log self-describing when inspected without joining against telemetry.

EventTelemetry. The `EventTelemetry` table extends each event with a complete snapshot of the controller’s internal state. Its 14 numeric columns capture the estimated pose (`state_x`, `state_y`, `state_theta`), the GPS reading (`gps_x`, `gps_y`), the control errors computed from the current pose to the goal (`error_distance`, `error_heading`), the current linear and angular velocity (`current_vel_linear`, `current_vel_angular`), the command velocities output by the control module (`target_vel_linear`, `target_vel_angular`), and the coordinates of the next waypoint on the low-level path (`next_point_x`, `next_point_y`). The waypoint fields are nullable because the planner may produce an empty path under certain conditions; all other fields are non-nullable `DOUBLE` columns.

A `UNIQUE` constraint on (`sim_id`, `event_time`, `e_type`) mirrors the `Events` composite primary key, allowing each event to have at most one telemetry row. The relationship is therefore one-to-zero-or-one from `Events` to `EventTelemetry`: a telemetry row must reference an event, while the schema does not require every event to have telemetry. A foreign key from (`sim_id`, `event_time`, `e_type`) to the corresponding columns in `Events` enforces this referential integrity.

3.3 Data Flow and Controller Integration

The logging lifecycle is tightly coupled to the controller’s execution in `AGVSimulation.run()` (`DefaultController.py:73`). During initialisation, the constructor instantiates a `RobotLog` with the configured server URL and the robot’s `unit_id`, then calls `logger.start()`. This method sends a `POST /start` request and, upon success, stores the returned `sim_id` and logs a `START` event on the client side. If the server does not respond successfully within ten attempts, spaced one second apart, the method returns `False`. The controller responds by calling `_refuse_to_run()`, which stops the motors and leaves the robot immobile for the remainder of the simulation. This design ensures that no robot operates without a persistent audit trail.

Once the simulation is running, the controller’s main loop executes at the Webots step rate. On every cycle, three logging operations take place:

1. `capture_telemetry()` (`robot_log.py:101`) stores the latest state estimate, GPS reading, control command and low-level waypoint in the `latest_telemetry` dictionary. This snapshot is held in memory and attached to the next event that is logged.
2. `update_obstacle_state()` (`robot_log.py:149`) compares the current obstacle presence flag from the low-level planner’s safety status with the previous state. A transition from clear to obstructed produces an `OBSTACLE_ENCOUNTER` event; the reverse transition produces `OBSTACLE_CLEARED`. The obstacle count is incremented once per encounter, not per cycle, giving a meaningful count of distinct obstruction episodes.
3. `update_idle_state()` (`robot_log.py:163`) detects transitions based on whether the robot’s linear velocity magnitude falls below the threshold `idle_speed_threshold` (default 10^{-3}). It accumulates the total idle time and logs `IDLE_START` and `IDLE_END` events at each transition.

Additionally, `log_target_reached()` is called from the controller’s goal-handling logic (`DefaultController`) when the distance error to the current target drops below `goal_reached_thresh`. This produces a `REACHED_TARGET` event that includes the goal index and coordinates.

Rather than sending each event individually, the `RobotLog` accumulates events and their matching telemetry records in Python lists and flushes them in periodic batches. The flush

interval is controlled by `LogConfig.log_interval` in `config.py`, set to 5 seconds by default. The decision to batch events rather than stream them avoids saturating the HTTP server with a request per simulation step (which runs at hundreds of hertz) and reduces the number of database transactions.

The batch flush procedure in `RobotLog.flush()` (`robot_log.py:208`) uses a two-queue design. The `failed_events` and `failed_event_telemetry` lists hold records that could not be persisted in a previous attempt. On each flush, the failed queue is retried first by calling `_send_batch()`. If the retry succeeds, the failed queues are cleared and the current buffer is sent. If the retry fails, the current buffer is appended to the failed queues, preserving the chronological order of events across multiple failed attempts. Only after a successful round-trip are events removed from both the main and failed queues.

When the simulation terminates, the `finally` block of `run()` (`DefaultController.py:126`) calls `logger.stop()`, which logs a `STOP` event and enters a retry loop for flushing all remaining data, including the failed queue. If the server remains unreachable after ten attempts, the method prints the number of unsaved records and returns `False`, accepting controlled data loss at a well-defined shutdown boundary.

3.4 Validation and Transactional Integrity

The server enforces payload correctness before any data reaches the database. The validation function `validate_save_payload()` (`logging_server.py:81`) performs a comprehensive check of every field in the request body. Numeric fields must be finite; integer fields must be of type `int` and nonnegative; string fields must respect length limits; the `e_type` of every event and telemetry record must belong to the `ALLOWED_EVENT_TYPES` frozenset; and the `sim_id` embedded in each sub-object must match the top-level `sim_id`. The validator also constructs sets of event identities, defined as tuples of `(sim_id, time, e_type)`, and verifies that the identities from the `events` and `event_telemetry` arrays match exactly and contain no duplicates. This cross-consistency check catches misalignments between the two arrays before the database transaction begins.

When validation fails, the function raises `PayloadValidationError` with a message identifying the offending field. The HTTP handler catches this exception and returns a 400 response, allowing the controller to distinguish between malformed data (which cannot be fixed by retrying) and infrastructure failures (which may be transient).

The `PersistenceService.save()` method (`logging_server.py:227`) writes accepted data inside a single database transaction. It first acquires a row-level lock on the target `Simulations` row with `SELECT ... FOR UPDATE` and verifies that the stored `unit_id` matches the requesting unit. This ownership check prevents a controller from accidentally or maliciously overwriting another robot's simulation data. The method then updates the summary columns and inserts events and telemetry rows using `executemany()` with `ON DUPLICATE KEY UPDATE` clauses. The idempotent insert makes the operation safe to retry: if a previous flush succeeded but the response was lost, re-sending the same batch will not create duplicate rows.

If any database operation raises an exception during the transaction, the entire batch is rolled back and a `PersistenceError` is raised, which the HTTP handler maps to status 500. The `save()` method treats `PayloadValidationError` specially: it rolls back the transaction but does not wrap the exception, allowing the handler to return status 400. This separation of error classes keeps the controller informed about the nature of each failure.

Concurrency is managed at two levels. The `PersistenceService` uses a `threading.Lock` to serialise all calls to `save()`, ensuring that transactions from concurrent flush operations by the two robots do not interleave. The MySQL connection is not held open between flushes; instead, `_get_connection()` (`logging_server.py:187`) checks the liveness of any cached connection with `ping(reconnect=True)` and creates a new connection when needed. This lazy-reconnect strategy avoids the complexity of connection pooling for a workload where writes arrive no more

than every five seconds.

3.5 JSONL Mirror and Dashboard Integration

After every successful database commit, `rebuild_jsonl()` (`logging_server.py:328`) regenerates three flat files in the `logging/logs` directory: `simulations.jsonl`, `events.jsonl` and `event_telemetry.jsonl`. Each file contains one JSON object per line, with keys derived from the MySQL column names. The rebuild queries all rows from the database within a consistent read snapshot obtained by the transaction started in `save()`, guaranteeing that the JSONL files faithfully represent the committed state.

The files are written atomically using a temporary-file-and-rename pattern (`_write_jsonl_atomic()`, line 372). Data is written to a temporary file in the same directory, flushed to disk with `os.fsync()`, and then moved to the final destination with `os.replace()`, which is atomic on the same filesystem. If any exception interrupts the process, the temporary file is removed. A reader such as the web application described in section 4 therefore always sees a complete, self-consistent snapshot rather than partial content.

This mirror serves two purposes. First, it decouples the read path from the write path: the monitoring dashboard can poll the JSONL files without connecting to MySQL or competing for database locks, keeping the server's response time predictable. Second, it creates a durable, human-readable archive of every simulation run. A developer can inspect the latest simulation with `tail simulations.jsonl`, search for specific event types across all runs with `grep`, or import the data into a Python notebook for statistical analysis, without navigating the MySQL interface.

3.6 Reliability Considerations

The logging system adopts a conservative approach to reliability that prioritises data completeness over availability during startup while tolerating transient failures during steady-state operation.

The most consequential decision is the controller's refusal to move if the logging service cannot create a simulation record at startup, enforced in `RobotLog.start()` (`robot_log.py:79`). Ten consecutive HTTP failures are treated as a terminal condition; the robot remains immobile for the entire simulation. This stop-the-world posture is chosen because a missing simulation record would render all subsequent events unanchored. The `Events` and `EventTelemetry` tables carry foreign keys referencing `Simulations.id`, and without a valid `sim_id` no event data can be committed to the database. Buffering events indefinitely in hope of a late recovery would risk unbounded memory growth and opaque data loss, making post-hoc debugging harder.

During normal operation the batch-flush design trades some timeliness for robustness. Events spend up to five seconds in controller memory before being sent to the server. A crash of the controller process or Webots within a flush interval loses at most the events accumulated since the last successful flush. The failed-events queue absorbs transient server unavailability: if the server is down when a batch is attempted, the records are preserved and retried on the next interval without interrupting the simulation loop. This decoupling ensures that a brief database outage does not halt the robots, at the cost of potentially losing the in-memory batch if the controller itself terminates before the next successful flush.

At shutdown, the `RobotLog.stop()` method makes up to ten additional flush attempts. Data that remains unsaved after these attempts is discarded, and a console message reports the count. This explicit best-effort-accept-loss boundary avoids blocking the application exit indefinitely while making the extent of any data loss visible to the operator.

The MySQL container is provisioned via Docker, as documented in the project `README.md`. The database connection parameters (host, port, user, password, database name) are read from environment variables with defaults that match the Docker setup, allowing the server to be

reconfigured without code changes. The container runs on the standard MySQL port 3306 and stores its data in a Docker volume that persists across container restarts.

3.7 Use in Evaluation

The structured event log is the primary source of quantitative evidence for evaluating controller behaviour. The `EventTelemetry` table records the robot's full estimated state at every logged moment, enabling offline reconstruction of trajectories, analysis of convergence toward goals, and comparison of planning decisions across different controller configurations or environment layouts.

The summary metrics aggregated per simulation run (`total_sim_time`, `total_idle_time`, `obstacle_count`, `event_count`) support direct quantitative comparison between runs. For example, total idle time can serve as a proxy for navigation efficiency: a robot that waits for obstacles to clear contributes more idle time than one that finds alternative routes. The obstacle count reflects the frequency of dynamic interference, which varies with the movement patterns of the supervisor-controlled humans and forklifts in the environment. Because each simulation row is identified by its `unit_id`, results from the two robots operating concurrently can be compared side by side, controlling for the same world conditions.

The `details` column of the `Events` table provides a lightweight narrative of each run. Searching for `OBSTACLE_ENCOUNTER` events reveals the locations and frequencies of obstruction episodes; inspecting the GPS coordinates in the details string can identify bottlenecks in the warehouse layout where dynamic obstacles cluster. `UNEXPECTED_BEHAVIOR` events capture anomalous conditions that the controller explicitly flags, making them immediately searchable without scanning the full telemetry.

The JSONL mirror enables ad-hoc analysis with standard command-line tools. A developer can extract all events of a given type with `grep`, compute aggregate statistics from the telemetry file with a short script, or load the data into a Jupyter notebook for more involved analysis. The web application described in section 4 consumes the same JSONL files to render the monitoring dashboard, ensuring that the same data source serves both programmatic inspection and visual oversight.

4 Web Application

The web application provides an operational view of the AGV simulations. It combines the data exposed by the logging pipeline with a visual representation of the warehouse, allowing users to inspect vehicle status, configured goals and the progression of a run.

4.1 Interface Design

Describe the principal views of the application and the information each view presents. Explain how the interface supports the workflow of monitoring a live simulation and reviewing a completed one.

4.2 Data Integration

Explain how the web application obtains and transforms simulation data before rendering it. Mention refresh behaviour, data formats and any consistency constraints that affect what users see.

4.3 Practical Value and Limitations

Summarize the value of the dashboard for development and demonstrations. Then state the current limitations, such as dependence on the logging service or any incomplete visualizations,

and identify the most useful future improvements.